# THE UNIVERSITY OF AZAD JAMMU AND KASHMIR

**Department of Software Engineering**

---

# DIGITAL IMAGE PROCESSING LAB

**Lab # 01**

***Python Setup, Fundamentals, and Imports***


**SUBMITTED BY**

Shahzad Ahmed Awan

Roll No. 2024-SE-15


**SUBMITTED TO**

Engr. Fahad Nisar

Lab Instructor

**SEMESTER AND SESSION**

Fifth Semester

Session 2024–28 (SE)

**DATE OF SUBMISSION**

October 23, 2026

# I. Introduction

Digital image processing treats an image as a numerical matrix or two-dimensional array that can be manipulated and analyzed using programming code. Python is the standard language for digital image processing due to its clear syntax and powerful scientific packages. Before implementing image processing algorithms, a reliable programming workspace must be established and verified.

This laboratory report documents the setup of the Python development workspace and reviews fundamental programming syntax required for subsequent lab tasks. The core objectives are to:

1. Install Python 3 and Visual Studio Code (VS Code) and verify environment configuration;
2. Write and execute a basic Python script (hello.py) from the command line;
3. Practice basic syntax, conditions, loops, functions, and data collections in basics.py;
4. Install core image processing libraries (NumPy, Pillow, Matplotlib) and verify required import statements.

# II. Environment and Tools

All practical work was conducted on a Windows computer using the following setup:

- **Interpreter:** Python 3.14.8 (64-bit), installed at C:\Python314\python.exe.
- **Editor / IDE:** Visual Studio Code with the official Python extension by Microsoft.
- **Terminal:** Windows Command Prompt (cmd.exe) and the VS Code integrated terminal (PowerShell).
- **Project Folder:** dip-lab-01 containing hello.py, basics.py, and lab01_imports.py.
- **Packages:** NumPy (array computing), Pillow (image processing tools), and Matplotlib (data plotting).

# III. Procedure and Results

## A. Python Installation and Verification (Steps 1 & 2)

Python 3 was downloaded from the official website and installed with 'Add Python to PATH' selected. A new Command Prompt window was opened to verify the Python interpreter and pip package manager versions:

```
python --version
python -m pip --version
```

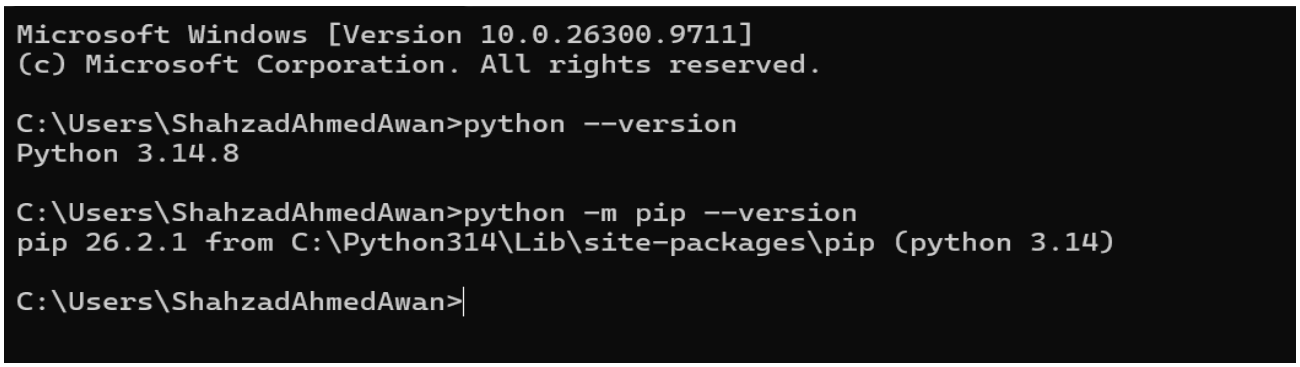



**Fig. 1.** *Python and pip version checks in Command Prompt.*

## B. VS Code Setup and Interpreter Selection (Step 3)

Visual Studio Code was installed along with the official Python extension by Microsoft. Pressing Ctrl + Shift + P opened the Command Palette, where 'Python: Select Interpreter' was chosen to set the active Python 3.14.8 interpreter.

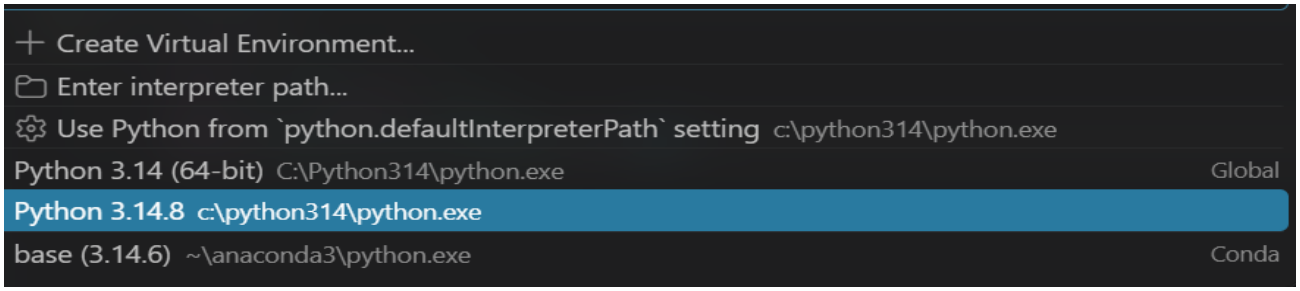



**Fig. 2.** *Select Interpreter dialog in Visual Studio Code showing active Python 3.14.8 executable.*

## C. First Program: hello.py (Step 4)

Created hello.py in VS Code inside the dip-lab-01 directory and saved the program code. Executed the program from terminal:

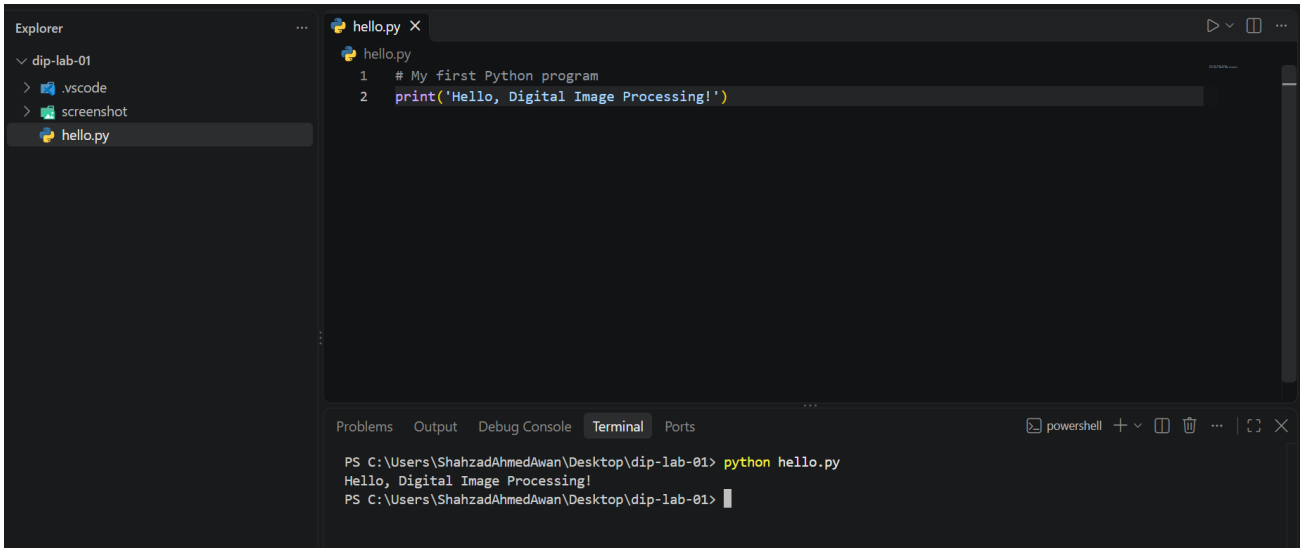



**Fig. 3.** *hello.py source code in VS Code editor and execution output in terminal.*

## D. Basic Python Fundamental Exercises: basics.py (Step 5)

Created basics.py to complete all six basic exercises. Each exercise section is presented below with its complete source code script, theoretical/technical explanation, code editor screenshot, and terminal execution output screenshot:

### *1. Exercise 1: Variables and Data Types*

In Exercise 1, four variables representing image metadata were initialized: width (integer), height (integer), image_name (string), and is_color (boolean). The value and dynamic data type of each variable were retrieved using Python's built-in type() function. The total pixel count of the image canvas was calculated by computing the product of width * height (640 * 480), yielding the expected 307,200 pixels.

```python
# ===== Exercise 1: Variables and Data Types =====
print_separator("Exercise 1: Variables & Data Types")
width = 640
height = 480
image_name = "sample.png"
is_color = True

print(f"Variable 'width'      : {width}         \t| Type: {type(width)}")
print(f"Variable 'height'     : {height}        \t| Type: {type(height)}")
print(f"Variable 'image_name' : {image_name}    \t| Type: {type(image_name)}")
print(f"Variable 'is_color'   : {is_color}      \t| Type: {type(is_color)}")

total_pixels = width * height
print(f"\nTotal calculated pixels (width * height): {total_pixels}")
```

**Fig. 4.** *Exercise 1: Source code implementation in VS Code.*

```
--------------- Exercise 1: Variables & Data Types ---------------
Variable 'width'      : 640             | Type: <class 'int'>
Variable 'height'     : 480             | Type: <class 'int'>
Variable 'image_name' : sample.png      | Type: <class 'str'>
Variable 'is_color'   : True            | Type: <class 'bool'>

Total calculated pixels (width * height): 307200
```

**Fig. 5.** *Exercise 1: VS Code integrated terminal output displaying variable values, types, and total pixels.*

### *2. Exercise 2: if, elif, and else (Pixel Intensity Thresholding)*

Exercise 2 evaluates 8-bit grayscale pixel intensity values (ranging from 0 to 255). A conditional structure checks boundary conditions first: values below 0 or above 255 are classified as 'Invalid'. Valid pixel values of 128 or above are categorized as 'Bright pixel', while values below 128 are labeled as 'Dark pixel'. Testing boundary cases (-1, 0, 127, 128, 255, 256) confirmed correct logic flow across all threshold edges.

```python
# ===== Exercise 2: if, elif, and else =====
print_separator("Exercise 2: if, elif, and else")
test_intensities = [-1, 0, 127, 128, 255, 256]

for intensity in test_intensities:
    if intensity < 0 or intensity > 255:
        label = "Invalid"
    elif intensity >= 128:
        label = "Bright pixel"
    else:
        label = "Dark pixel"
    print(f"Intensity {intensity:3} : {label}")
```

**Fig. 6.** *Exercise 2: Conditional intensity logic source code.*

```
--------------- Exercise 2: if, elif, and else ---------------
Intensity  -1 : Invalid
Intensity   0 : Dark pixel
Intensity 127 : Dark pixel
Intensity 128 : Bright pixel
Intensity 255 : Bright pixel
Intensity 256 : Invalid
```

**Fig. 7.** *Exercise 2: Terminal output verifying boundary testing results.*

### *3. Exercise 3: for and while Loops*

Exercise 3 demonstrates iteration mechanisms in Python. A for loop iterates over a discrete sequence of intensity values [20, 130, 255], evaluating each value against a threshold of 128 to print 'Dark pixel' or 'Bright pixel'. Following this, a while loop manages an explicit loop counter variable, printing values 0, 1, and 2 before terminating when the condition counter < 3 evaluates to False.

```python
# ===== Exercise 3: for and while Loops =====
print_separator("Exercise 3: for and while Loops")
# for loop: visit each value in the list
print("For loop (threshold = 128):")
for value in [20, 130, 255]:
    if value >= 128:
        print(f"  Value {value:3} : Bright pixel")
    else:
        print(f"  Value {value:3} : Dark pixel")

# while loop: runs until counter reaches 3
print("\nWhile loop (counts 0 to 2):")
counter = 0
while counter < 3:
    print(f"  Counter : {counter}")
    counter += 1  # update the counter so the loop stops
```

**Fig. 8.** *Exercise 3: Iteration loops source code.*

```
--------------- Exercise 3: for and while Loops ---------------
For loop (threshold = 128):
  Value  20 : Dark pixel
  Value 130 : Bright pixel
  Value 255 : Bright pixel

While loop (counts 0 to 2):
  Counter : 0
  Counter : 1
  Counter : 2
```

**Fig. 9.** *Exercise 3: Loop iteration output in terminal.*

### *4. Exercise 4: Functions with Default Parameters*

Exercise 4 defines a modular user-defined function named is_bright(pixel, threshold=128). The function accepts a mandatory pixel value and an optional threshold parameter defaulted to 128. It evaluates pixel >= threshold and returns a boolean value (True or False). Calling is_bright(127) returns False, is_bright(128) returns True, and overriding the threshold with is_bright(200, 210) returns False as expected.

```
# ===== Exercise 4: Functions =====
print_separator("Exercise 4: Functions")


def is_bright(pixel, threshold=128):
    """Return True if pixel is at least the threshold, otherwise False."""
    return pixel >= threshold


print(f"is_bright(127)      : {is_bright(127)}")
print(f"is_bright(128)      : {is_bright(128)}")
print(f"is_bright(200, 210) : {is_bright(200, 210)}")
```

**Fig. 10.** *Exercise 4: Function definition and invocation code.*

```
--------------- Exercise 4: Functions ---------------
is_bright(127)      : False
is_bright(128)      : True
is_bright(200, 210) : False
```

**Fig. 11.** *Exercise 4: Boolean return values in terminal.*

### *5. Exercise 5: Lists, Tuples, and Detailed Mutability Analysis*

Exercise 5 explores sequence data structures and memory mutability in Python. A list pixels = [0, 128, 255] was created, accessed by index (pixels[0] = 0), modified in-place by updating pixels[1] = 150, and extended using pixels.append(64) to produce [0, 150, 255, 64] with length 4. In addition, an immutable tuple size = (640, 480) was created and unpacked into width and height variables.

```
# ===== Exercise 5: Lists and Tuples =====
print_separator("Exercise 5: Lists and Tuples")
# List operations
pixels = [0, 128, 255]
print(f"Original list       : {pixels}")
print(f"First item          : {pixels[0]}")

pixels[1] = 150
pixels.append(64)
print(f"Modified list       : {pixels}")
print(f"Total list length   : {len(pixels)}")

# Tuple operations
size = (640, 480)
width, height = size
print(f"\nUnpacked Tuple      : Width = {width}, Height = {height}")

# Explanation: A list is mutable, meaning its items can be reassigned, added, or removed.
# A tuple is immutable, meaning its structure and items are permanently fixed upon creation.
```

**Fig. 12.** *Exercise 5: List and tuple mutability code implementation.*

```
--------------- Exercise 5: Lists and Tuples ---------------
Original list       : [0, 128, 255]
First item          : 0
Modified list       : [0, 150, 255, 64]
Total list length   : 4

Unpacked Tuple      : Width = 640, Height = 480
```

**Fig. 13.** *Exercise 5: Output confirming list modifications and tuple unpacking.*

**Detailed Explanation of Mutability (List vs. Tuple):**

• List Mutability: A list in Python is a mutable data structure stored in dynamic memory. Its elements, size, and order can be altered in-place after creation (via index assignment, appending, or removing elements) without modifying the memory address of the list object itself.

• Tuple Immutability: A tuple is an immutable data structure. Once instantiated in memory, its elements and size are fixed permanently. Attempting to modify or reassign a tuple item (e.g., size[0] = 800) raises a TypeError. In digital image processing, fixed dimensions like (width, height, channels) are stored as tuples to guarantee integrity, while dynamic pixel processing arrays use mutable lists or NumPy arrays.

### *6. Exercise 6: Dictionaries and Sets*

Exercise 6 covers key-value mappings and unique collection sets. A dictionary image = {'width': 640, 'height': 480} was created, extended with key 'mode': 'RGB', and queried by key name. A list containing duplicate strings ['cat', 'dog', 'cat', 'bird'] was passed into set(), which automatically filters out duplicate elements, returning 3 unique items. Sets are unordered, unique hash tables highly suitable for storing unique class labels in computer vision.

```python
# ===== Exercise 6: Dictionaries and Sets =====
print_separator("Exercise 6: Dictionaries and Sets")
# Dictionary operations
image = {"width": 640, "height": 480}
print(f"Original dictionary : {image}")

image["mode"] = "RGB"
print(f"Modified dictionary : {image}")
print(f"Width (by key)      : {image['width']}")
print(f"Mode (by key)       : {image['mode']}")

# Set operations
label_list = ["cat", "dog", "cat", "bird"]
labels = set(label_list)
print(f"\nOriginal list       : {label_list}")
print(f"Unique labels count : {len(labels)}")
# Note: a set removes duplicates, and its display order may vary.
```

**Fig. 14.** *Exercise 6: Dictionary and set source code.*

```
--------------- Exercise 6: Dictionaries and Sets ---------------
Original dictionary : {'width': 640, 'height': 480}
Modified dictionary : {'width': 640, 'height': 480, 'mode': 'RGB'}
Width (by key)      : 640
Mode (by key)       : RGB

Original list       : ['cat', 'dog', 'cat', 'bird']
Unique labels count : 3
```

**Fig. 15.** *Exercise 6: Unique element count and dictionary value output.*

## E. Package Installation and Verification (Step 6)

The core scientific image processing packages were installed using python -m pip install numpy pillow matplotlib. Executing pip show confirmed package installations, versions, and local site-packages directory paths:

```
python -m pip install numpy pillow matplotlib
python -m pip show numpy pillow matplotlib
```

```
Downloading pillow-13.0.0-cp314-cp314-win_amd64.whl (7.3 MB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 7.3/7.3 MB 401.6 kB/s  0:00:18
Downloading matplotlib-3.11.2-cp314-cp314-win_amd64.whl (9.5 MB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 3.4/9.5 MB 2.9 MB/s eta 0:00:03
```

**Fig. 16.** *Installation progress of NumPy, Pillow, and Matplotlib via pip.*

```
Problems  Output  Debug Console  Terminal  Ports

PS C:\Users\ShahzadAhmedAwan\Desktop\dip-lab-01> python -m pip show numpy pillow matplotlib | findstr /B /C:"Name:" /C:"Ver
sion:" /C:"Location:"
Name: numpy
Version: 2.5.4
Location: C:\Users\ShahzadAhmedAwan\AppData\Roaming\Python\Python314\site-packages
Name: pillow
Version: 13.0.0
Location: C:\Users\ShahzadAhmedAwan\AppData\Roaming\Python\Python314\site-packages
Name: matplotlib
Version: 3.11.2
Location: C:\Users\ShahzadAhmedAwan\AppData\Roaming\Python\Python314\site-packages
PS C:\Users\ShahzadAhmedAwan\Desktop\dip-lab-01>
```

**Fig. 17.** *pip show output verifying installation details for NumPy, Pillow, and Matplotlib.*

## F. Import Statement Verification (Step 8)

Created lab01_imports.py containing required import statements with standard aliases. Executing python lab01_imports.py in the VS Code integrated terminal completed silently without error messages, verifying full

import functionality:

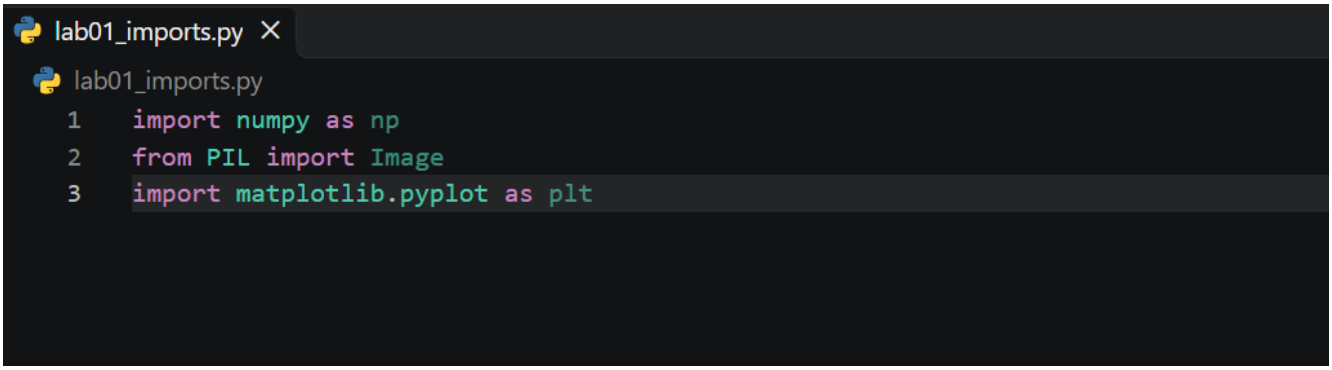



**Fig. 18.** *Saved import statements in lab01_imports.py.*

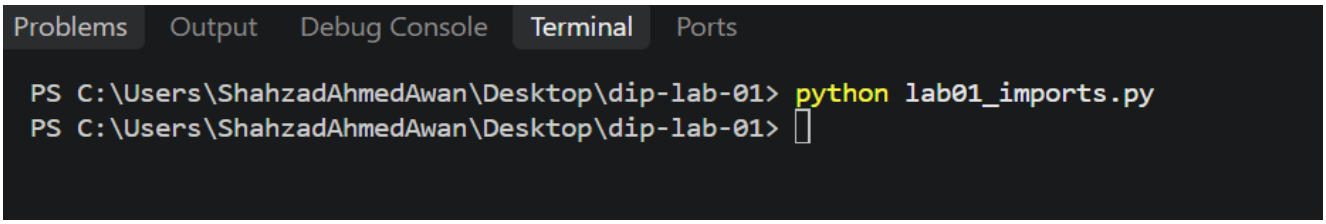



**Fig. 19.** *Successful, error-free execution of lab01_imports.py in the VS Code integrated terminal.*

## IV. Discussion and Conceptual Explanation

### A. Python versus pip, and Purpose of Import Statements

Python is the core high-level programming language interpreter responsible for executing code scripts and managing runtime operations. In contrast, pip (Pip Installs Packages) is Python's command-line package management tool used to fetch, install, upgrade, and manage external third-party libraries hosted on the Python Package Index (PyPI).

Import statements load external modules into an active script's namespace. In Digital Image Processing, import numpy as np loads multi-dimensional matrix operations essential for numerical pixel matrices, from PIL import Image provides file I/O and format conversion utilities, and import matplotlib.pyplot as plt enables graphical display, histogram plotting, and visual output rendering.

### B. Key Technical Observations

1. Intensity Threshold Boundary Checks: Testing boundary values (-1, 0, 127, 128, 255, 256) ensured robust edge-case validation for 8-bit image processing.
2. Data Structure Selection: Mutable lists allow dynamic pixel manipulation, immutable tuples guarantee fixed image canvas dimensions, dictionaries store metadata key-value attributes, and sets maintain unique object labels.
3. Environment Integrity: Invoking 'python -m pip' ensures package installation aligns directly with the active VS Code interpreter executable path.

## V. Conclusion

The Python development environment and Visual Studio Code IDE were successfully installed, configured, and verified on a Windows system. All basic syntax exercises (variables, threshold conditionals, loops, custom functions, lists, tuples, dictionaries, and sets) were completed in basics.py with verified outputs. Essential digital image processing packages (NumPy, Pillow, Matplotlib) were installed using pip, and lab01_imports.py executed with zero errors. The workspace is fully established and prepared for advanced digital image processing laboratory assignments.

## VI. Submitted Files

- **hello.py:** First Python verification script printing initialization string ('Hello, Digital Image Processing!').
- **basics.py:** Complete solutions for Exercises 1–6 covering variables, conditions, loops, functions, lists/tuples mutability, dictionaries, and sets.
- **lab01_imports.py:** Import statement verification script for NumPy, Pillow, and Matplotlib.